Chapter 11: Generic and Reflection

Outline

- Why generics? What is generic?
- Generic classes, generic methods, and generic interfaces
- Generic wildcards
- Generic design - template method pattern
- Reflection
- Design security of singleton pattern

Why do you need generics?

- Consider the following code for problems

```
ArrayList list = new ArrayList();
list.add("str");
list.add(100);
list.add(true);
String name = (String) list.get(0);
```

1. No error checking, you can add any object to the array list.
2. Forced type conversion must be done to get a value.

What is generic?

- Java generics are a new feature introduced in Java 5 that provides compile-time type safety monitoring that allows us to detect illegal data structures at compile time.
  - Introduce type parameters, which specifies the data type of all operations as a parameter.

For example, specify a generic String

```
ArrayList<String> list = new ArrayList<>();
list.add("str");
String name = list.get(0);
```

```
ArrayList list = new ArrayList<>();
list.add("str");
String name = (String) list.get(0);
```

When the get method is called, there is no need for forced type conversion, and the compiler knows that the return value is of type String, not Object.

What is generic?

- Consider whether the following is feasible

```
ArrayList<String> list = new ArrayList<>();
list.add(100);
```

- For no reason, the compiler prompts

```
Required type: String
Provided: int
```

The compiler knows<String> that the add method in List has a parameter of type String, which is safer than using the Object type. Illegal data types detected during the compilation phase.

Outline

- Why generics? What is generic?
- Generic classes, generic methods, and generic interfaces
- Generic wildcards
- Generic design - template method pattern
- Reflection
- Design security of singleton pattern

Generic Classes (1)

- A generic class is a class with one or more type variables.
  - Syntax format: Declare a generic after the class name, and the generic class declares the type variable before using it. Type variables usually use shorter capitalizations such as T, E, K, V, etc.

Declare a single generic

```
class Generic<T> {                  //Generic type of declarative class <T>
    public T t;                     //The type of variable t is generic T
    public Generic(){}
    public Generic(T t){            //The type of incoming parameter is T
        this.t = t;
    }
    public void fun1(T t) {}
    public T fun2(T t) {}           //The function returns a value of type T
}
```

Generic Classes (4)

- Multiple generic classes are used

```
class Generic<T, E> {
    public T t;
    public E e;
    public void fun(T t, E e) {}
}

Generic<String, Integer> g3 = new Generic<>();
g3.t = "str";
g3.e = 100;
```

- Note that generics cannot be used for basic data types, such as int

```
Generic<int> g = new Generic<>();
```

```
Generic<Integer> g = new Generic<>();
```

Generic Method (1)

- Generic methods can be defined in a normal class or in a generic class. A method that uses generic members in a generic class is not a generic method, only a method that declares a generic is a generic method.

Note that the following is not a generic method

```
class Generic<T> {
    public T t;
    public void func(T t) {}
}
```

✖ Generic methods are not explicitly declared

Note that the following is a generic method

```
class Generic<T> {
    public T t;
    public <T> void func(T t) {}
}
```

Generic Methods (2)

Generic methods are defined in the non-generic class

```
class GenericFun{
    public <T,E> void fun1(E e){}
    public <T> T fun2(T t){
        return t;
    }
}
```

- When calling a generic method, fill in the specific type in parentheses in the method name.

  GenericFun g = new GenericFun();
  g.<String>fun2("str");
- In most cases, method calls can omit type parameters.

  g.fun2("str");//The return value is str

Generic Methods (3)

Generic methods are defined in the generic class

```
class GenericFun<K>{
    public <T> T fun2(T t,K k){
        return null;
    }
}
```

- With the generic method, the incoming argument here must match the type declared by the generic class

```
GenericFun<Integer> g = new GenericFun<>();
g.fun2("str1",123) ;
```

Generic Methods (4)

- If the generic of a generic method matches the generic name declared by the generic class, the generic in the generic method overrides the generic of the class.

```
class GenericFun<K>{
    public <T,K> T fun2(T t, K k){
        return null;
    }
}
```

- Using the generic method, the generic of the class is Integer, and the parameters passed by the generic method are two Strings

```
GenericFun<Integer> g = new GenericFun<>();
g.fun2("str1","str2") ;
```

Generic Methods (5)

- The static generic method of the class must not use the generics declared in the generic class, and can be declared independently.

```
class GenericStaticMethod<K>{
    private K k;
    public GenericStaticMethod(K k){
        this.k=k;
}
    public static GenericStaticMethod <K> fun1(K k){
        return new GenericStaticMethod <K>(k)}
}//Error: Unable to reference non-static variable K from static context
    public static <K> GenericStaticMethod<K> fun1(K k){
        return new GenericStaticMethod<K>(k)}
}//Compilation is successful
     public static <T> GenericStaticMethod<T> fun1(T t){
        return new GenericStaticMethod<T>(t)}
}//Written in another type
```

Generic method

```
// Generic declarations are used when defining the class Durian
class Durian<T> {
    // Use the T-type parameter to define instance properties
    private T info;

    // The following method uses the T-type parameter to define the constructor
    public Durian(T info) { this.info = info; }
    public void setInfo(T info) { this.info = info;}
    public T getInfo() {   return this.info;}

    // Static generic methods should be distinguished using other types:
    public static <K> Durian<K>readyear(K info) {
        return new Durian<K>(info);
    }
}
```

Generic method

```
public class GenericDurian {
    public static void main(String[] args) {
        // Since the T-parameter is passed to String, the constructor parameter can only be String
        Durian<String> a1 = new Durian<>("Musang King");//name
        System.out.println(a1.getInfo());
        // Since the T-parameter is passed a Double, the constructor parameter can only be a Double
        Durian<Double> a2 = new Durian<>(1.23);//weight
        System.out.println(a2.getInfo());

        Durian<Integer> a3 = Durian.readyear(2025);
        System.out.println(a3.getInfo());
    }
}
```

```
public static <K> Durian<K>readyear(K info) {
        return new Durian<K>(info);
}
```

Generic Interfaces (1)

Define the following generic interfaces

```
interface GenericInterface <T>{
    T fun1();
}
```

The implementation class is a non-generic class, and the generic type of the interface must be specified

```
class GenericInterfaceImpl implements GenericInterface<String> {
    @Override
    public String fun1() {
        return null;
    }
}
```

Generic Interfaces(2)

Define the following generic interfaces

```
interface GenericInterface <T>{
    T fun1();
}
```

The implementation class is a generic class, and the generic of the implementation class should be consistent with the interface

```
class GenericInterfaceImpl<T> implements GenericInterface<T> {
    @Override
    public T fun1() {
        return null;
    }
}
```

Generic interfaces

```
interface ShowInterface<T> { public void show(T t); }

//The implementation class is non-generic
class ShowClass1 implements ShowInterface<String>{
      public void show(String t){
            System.out.println(“show:”+t);
      }
}

//The implementation class is generic
Class ShowClass2<T> implements ShowInterface<T>{
      public void show(T t){
            System.out.println("show:"+t);
      }
}
```

Generic interfaces

```
public static void main(String[] args) {
	//The implementation class is non-generic
	ShowClass1 obj = new ShowClass1();
	obj.show("java");

	//The implementation class is generic, type is determined when used
	ShowClass2<Integer> obj = new ShowClass2<>();
	obj.show(6);
}
```

Generic wildcards

- A strict generic type system cannot be changed once specified
- You need to allow type parameters to change
- At the same time, you need to control the types that can be specified, rather than being unlimited

Generic wildcards

- wildcard
  - Allow type parameters to change
  - <? extends ClassName>: The type argument is a subclass of the ClassName
  - <? super ClassName>: The type parameter is a superclass of ClassName
  - <?>: No limit wildcard

Upper Bound Wildcard

- <? extends T>
  - The class must be a T class, or a subclass of T class

```
public class WildCardExtendsDemo {
    public static void printAllObject(ArrayList<? extends Number> list) {
        for (Object object : list) {
            System.out.println(object);
        }  }
    public static void main(String[] args) {
        ArrayList<Double> list1 = new ArrayList<>();
        list1.add(1.23);
        printAllObject(list1);
    }  }
```

Lower Bound Wildcard

- <? super T>
  - The class must be a T class, or a superclass of T class

```
public class WildCardSuperDemo {
    public static void printAllObject(ArrayList<? super Double> list) {
        for (Object object : list) {
            System.out.println(object);
        } }
    public static void main(String[] args) {
        ArrayList<Number> list1 = new ArrayList<>();
        list1.add(7);
        printAllObject(list1);
    } }
```

Type parameter T vs. wildcard?

- T represents a definite type
  - It is commonly used for the definition of generic classes and generic methods
- Wildcard indicates an indeterminate type, not a type variable
  - Commonly used for calling codes and parameters for generic methods, not for defining classes and generic methods.

```
T t = operate();
? a = operate();    //illegal
```

Template method patterns

Template Method

- Define the algorithm skeleton in an operation
- Defer some steps into the subclass
- The template method pattern allows subclasses to redefine certain steps of an algorithm without changing its structure

Template method patterns

- AbstractClass
  - Abstract templates, define and implement a template method
  - Give the skeleton of the top-level logic
- ConcreteClass
  - Implement one or more abstract methods defined by the parent class
- An abstract class can have any number of concrete subclasses
- Each abstract class can give different implementations of abstract methods

```
public abstract class AbstractClass
    {
        public abstract void PrimitiveOperation1();
        public abstract void PrimitiveOperation2();

        public void TemplateMethod()
        {
            PrimitiveOperation1();
            PrimitiveOperation2();
            System.out.println("");
        }
    }
```

Some abstract behaviors are placed in subclass implementations

The template method gives the skeleton of the logic, which is made up of some corresponding abstract operations, all of which are deferred to the subclass implementation

```
public class ConcreteClassA extends AbstractClass
   {
       public void PrimitiveOperation1()
       {
           System.out.println("Specific class A method 1 implementation");
       }
       public void PrimitiveOperation2()
       {
           System.out.println("Specific class A method 2 implementation");
       }
   }

   public class ConcreteClassB extends AbstractClass
   {
       public void PrimitiveOperation1()
       {
           System.out.println("Specific class B method 1 implementation");
       }
       public void PrimitiveOperation2()
       {
           System.out.println("Specific class B method 2 implementation");
       }
   }
```

Template method patterns

- Move the common behavior to the superclass to remove the duplicate code in the subclass
- Improved code reuse
- Applications
  - Implement the common part of an algorithm at once, leaving the different part to the subclass to implement
  - The behavior common in each subclass should be extracted and centralized into a common parent class to avoid code duplication
  - Control subclass expansion: The template method only calls the "hook" operation at specific points
  - (Hook operation: Provides default behavior, subclasses can be extended if necessary)

Template method patterns

- participator
  - Abstract and concrete classes
- collaboration
  - Concrete classes rely on abstract classes to implement common steps in algorithms
  - Abstract classes rely on concrete classes to implement the specific details of the algorithm
- effect
  - Template methods are especially important in libraries, where they extract public behavior from the library

Template method patterns

- The template approach results in a reverse control structure
  - Refers to the action of the parent class calling a subclass, not the other way around
  - Sometimes referred to as the "Hollywood Rule," i.e. "Don't look for us." We are looking for you"
- note
  - The template method must specify which operations are hook operations (which can be redefined) and which are abstract operations (which must be redefined)
  - Minimize the number of abstract operations that must be redefined when a subclass implements the algorithm

Reflection

```
// In the above example, create1 can be rewritten as
public static X create2(String name) {
    Class<?> class = Class.forName(name);
    X x = (X) class.newInstance();
    return x;
}
```

Pass the package name and class name to the create2() method, dynamically load the specified class through the reflection mechanism, and then instantiate the object

The above function of dynamically obtaining information and dynamically calling objects is called the reflection mechanism of the Java language

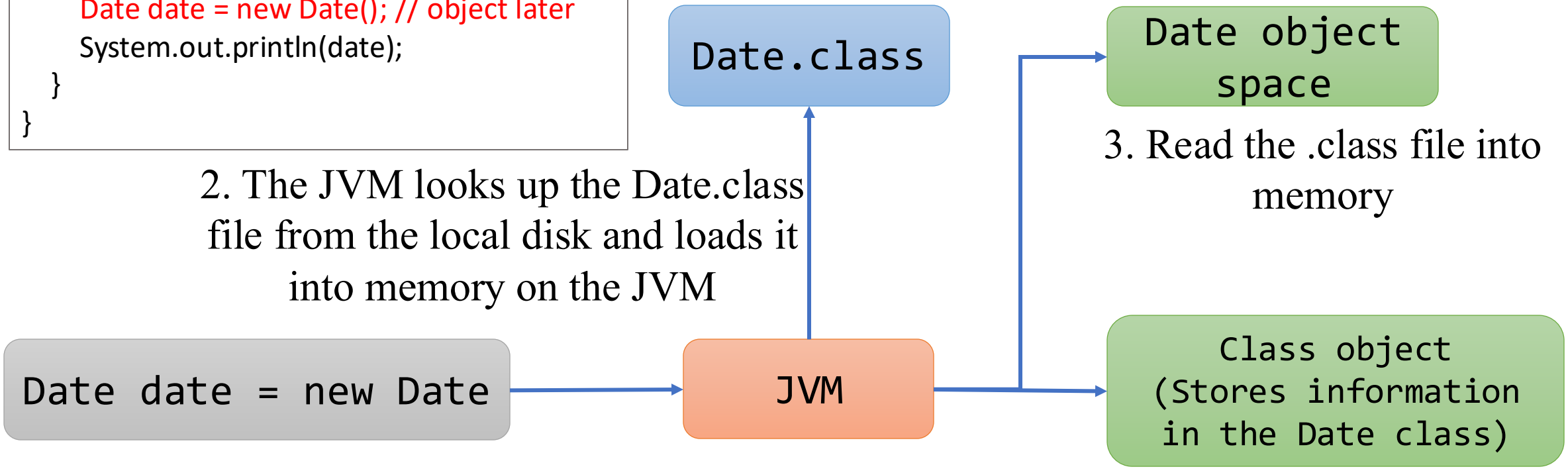
- When to reflect
  - Know the class to which any object belongs at runtime.
  - Construct an object of any class at runtime.
  - Know the member variables and methods that any class has at runtime.
  - Call the methods and properties of any one object at runtime.

Class

- The Java runtime system always maintains a runtime type identifier for all objects
- It keeps track of the complete structure information of the class to which each object belongs, including package names, class names, implemented interfaces, methods and fields owned, etc., and the JVM uses this information to select the correct method to execute
- This information can be accessed using special Java classes
- Understanding of Class classes
  - The Class class can be understood as a type of class
  - A Class object is called a class type object

Class

```
import java.util.Date; // class first
public class Test {
   public static void main(String[] args) {
      Date date = new Date(); // object later
      System.out.println(date);
   }
}
```

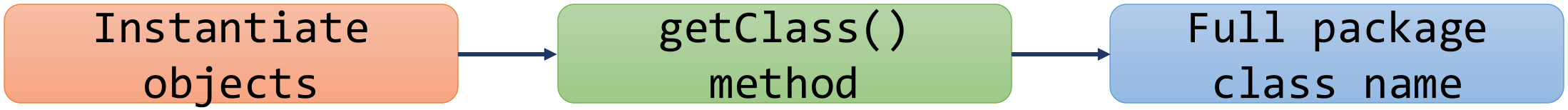


Class

- Normal way

```
Date date = new Date();
```

- Reflection mode




```
System.out.println(date.getClass()); // "class java.util.Date
```

Obtaining Class Objects(1)

- getClass() method
  - The getClass() method in the Object class returns an instance of the Class type

```
public class getClassTest {
    public static void main(String[] args) {
        Student Harry = new Student(“Harry Potter”,11); // The Student class describes the student
        System.out.println(Harry.getClass()); // The Class object describes the properties of a particular class
        // output Class Student
        System.out.println(Harry.getClass().getName());//Class.getName() Returns the name of the class
        // output Student
    }
}
```

Obtaining Class Objects(2)

- Class.forName()
  - Save the class name in a string
- T.class
  - T is an arbitrary Java type (may or may not be a class)

```
public class forNameTest {
    public static void main(String[] args) throws ClassNotFoundException {
        String className = "Reflect.Student";
        Class cl1 = Class.forName(className); //Class.forName
        Class cl2 = Student.class; //T.class
    }
}
```

Construct an instance of the class by reflection

Method 1: Use Class.newInstance

- The newInstance method calls the default constructor (parameterless) to initialize the newly created object
- If the class does not have a default constructor, an exception is thrown

class has no parameterless constructor,
What if the class constructor is private?

```
Date date1 = new Date();
Class dateClass2 = date1.getClass();
Date date2 = dateClass2.newInstance();
```

Construct an instance of the class by reflection

- Method 2: Use Constructor's newInstance
  - The construction method is obtained first through reflection and then called
  - Get the constructor first, and then execute the constructor
- distinguish
  - Constructor.newInstance can carry parameters
  - Class.newInstance is parameterless

Construct an instance of the class by reflection

Get the constructor

- Get all the "public" construction methods
  - public Constructor[] getConstructors() { }
- Get all constructs (including private, protected, default, public)
  - public Constructor[] getDeclaredConstructors() { }
- Get a "public" construction method that specifies the parameter type
  - public Constructor getConstructor(Class... parameterTypes) { }
- Get a "construction method" for the specified parameter type, which can be private, protected, default, or public
  - public Constructor getDeclaredConstructor(Class... parameterTypes) { }

Construct an instance of the class by reflection

- Use Constructor's newInstance to construct an instance of the class

```
public class ConTest {
    public static void main(String[] args) {
        Student Harry = new Student("Harry Potter", 11);
        Class StudentClass = Harry.getClass();
        Constructor con = StudentClass.getConstructor(String.class, int.class);
        Student Ron = (Student)con.newInstance("Ron Weasley", 11);
        System.out.println(Ron);
    }
}
```

Member variables are acquired and modified by reflection

Get and modify member variables

- Get all publicly owned fields
  - public Field[] getFields() { }
- Get all fields (including private, protected, default)
  - public Field[] getDeclaredFields() { }
- Get a public field with a specified name
  - public Field getField(String name) { }
- Get a field with a specified name, which can be private, protected, default
  - public Field getDeclaredField(String name) { }
- Use the get method in the Field class to view the field values
- Use the set method in the Field class to modify the field value

Member variables are acquired and modified by reflection

```
public class FieldTest {
    public static void main(String[] args) {
        Student Harry = new Student("Harry Potter", 11);
        Class StudentClass = Harry.getClass();
        Field f = StudentClass.getDeclaredField("name");
        f.setAccessible(true);
        Object v1 = f.get(Harry);
        System.out.println(v1);
        f.set(Harry,"The boy who lived");
        System.out.println(Harry.getName());
    }
} //Calling f.set(Harry, newValue) can modify the attribute of the harry object to newValue
```

Acquisition of members by reflection

Acquisition of members by reflection

- Get all "public methods" (including the parent class and of course the Object class)
  - public Method[] getMethods() { }
- Get all member methods, including private (excluding inherited)
  - public Method[] getDeclaredMethods() { }
- Get a member method that specifies the method name and parameter type
  - public Method getMethod(String name, Class<?>... parameterTypes)

Call the member method of the reflection acquisition

- Object invoke(Object obj, Object… args)
  - The first argument is which object is going to call this method
  - The first parameter is the argument passed when the method is called
  - For static methods, the first parameter can be ignored and set to null

```
public class MethodTest {
    public static void main(String[] args) {
        Student Harry = new Student("Harry Potter", 11);
        Method m = Student.class.getMethod("getName");
        String n = (String) m.invoke(Harry);
    }
}
```

Reflection

advantage

- It is more flexible and can dynamically obtain instances of classes at runtime

disadvantage

- Performance bottlenecks: Reflection is equivalent to a series of interpretation operations that tell the JVM what to do, and the performance is much slower than direct Java code
- Security issues: The reflection mechanism breaks encapsulation, as the class's private methods and fields can be obtained and called through reflection.

Singleton

- The purpose of a singleton is to ensure that there is only one instance of a class
  - Instantiate an object when the class is loaded

```
public class Singleton {
   private static Singleton singleton = new Singleton();
   private Singleton(){}

   public static Singleton getInstance(){
      return singleton;
   }
}
```

Breaks singleton using reflection

- Override access control in Java by setAccessible(true) in reflection, and then call a private constructor

```
public class SingletonTest {
    public static void main(String[] args) {
        Class objectClass = Singleton.class;
        Constructor constructor = objectClass.getDeclaredConstructor();
        constructor.setAccessible(true);
        Singleton instance = Singleton.getInstance();
        Singleton newInstance = (Singleton)constructor.newInstance();
        System.out.println(instance == newInstance);
    }
}// instance and newInstance are different instances of False
```

Resists reflection damage

```
public class Singleton {
    private static Singleton singleton = new Singleton();
    private Singleton(){
        if(singleton != null) {
            throw new RuntimeException("Singleton constructor prohibits calls by reflection");
        }
    }
    public static Singleton getInstance(){
        return singleton;
    }
}
```